

T15 — Tipi Generici, Wildcard e Type Erasure

Polimorfismo Parametrico, Bounded Type Parameters, Invarianza, Wildcard (PECS) e Type Erasure

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

0.1 1. Analisi Teorica Approfondita (Slide T15)

La lezione 15 introduce formalmente i **Tipi Generici** (*Generics*), introdotti in Java 5 per abilitare il **polimorfismo parametrico**, eliminare la fragilità dei cast a `Object` e garantire la **Type Safety** a tempo di compilazione.

0.1.1 1.1 L'Idea Guida & La Tazza `Cup<T>` (Slide 2-3)

Nella Slide 3 campeggia l'immagine di una tazza con la scritta:

`Cup<T>`

accompagnata dal motto: **“Same container, different content”**. Un contenitore (una tazza, una coppia, una lista, un albero) possiede una logica strutturale identica a prescindere dal tipo di dato che custodisce (caffè, tè, stringhe, interi, date). Prima di Java 5, per ottenere contenitori riusabili si usava `Object`, con conseguenze disastrose sull'affidabilità del software.

0.1.2 1.2 Polimorfismo Parametrico vs Approccio Basato su `Object` (Slide 4-7)

Consideriamo la funzione identità matematica $\lambda x. x$: * Se riceve un `int`, restituisce lo stesso `int`. * Se riceve una `String`, restituisce la stessa `String`.

L'approccio ingenuo (pre-Java 5): segnaposto `Object`*

```
1 public class Pair {
2     private Object first, second;
3     public Pair(Object first, Object second) {
4         this.first = first;
5         this.second = second;
6     }
7     public Object getFirst() { return first; }
8     public Object getSecond() { return second; }
9 }
```

- **Casting obbligatorio:**

```
1 Pair p = new Pair("one", "two");
2 String first = (String) p.getFirst(); // Cast esplicito verboso e tedioso
```

- **Il disastro della mancanza di Type Safety:**

```
1 Pair q = new Pair(1, 2); // Autoboxing da int a Integer
2 String second = (String) q.getSecond(); // COMPILA SENZA ERRORI!
3 // Ma a RUNTIME crasha con: java.lang.ClassCastException: class java.lang.Integer cannot be
  cast to class java.lang.String
```

0.1.3 1.3 Classi e Metodi Generici (Slide 8-10)

Con i Generics, la classe introduce una o più **variabili di tipo** (Type Parameters):

```
1 public class Pair<T> {
2     private T first, second;
3     public Pair(T first, T second) {
4         this.first = first;
5         this.second = second;
6     }
7     public T getFirst() { return first; }
8     public T getSecond() { return second; }
9 }
```

- **Sintassi d'uso:**

```
1 Pair<String> p = new Pair<String>("one", "two");
2 Pair<Integer> q = new Pair<Integer>(1, 2);
```

- **Controllo a compile-time:**

```
1 String first = p.getFirst(); // Nessun cast necessario: p restituisce String garantita!
2 String second = q.getSecond(); // COMPILE-TIME ERROR: Type mismatch (Integer non convertibile
  a String)
```

L'errore viene intercettato prima ancora di mandare in esecuzione il programma!

- **Regola fondamentale sui Tipi Primitivi: Un parametro di tipo T può essere sostituito solo da tipi riferimento (oggetti), MAI da tipi primitivi!** Non è consentito scrivere `Pair<int>`, ma si deve usare il wrapper `Pair<Integer>`.
- **Convenzioni di Naming:** Si usano lettere maiuscole singole:
 - T : Type generico.
 - E : Element (usato nelle collezioni: `List<E>`, `Set<E>`).
 - K , V : Key e Value (usato nelle mappe: `Map<K, V>`).
 - N : Number.
 - S , U : Tipi secondari ausiliari.

0.1.4 1.4 Interfacce della Libreria Standard Riscritte con i Generics (Slide 11-13)

- **Comparable<T>:**

```
1 public interface Comparable<T> {
2     int compareTo(T obj);
3 }
```

Consente alla classe che la implementa (es. `Student implements Comparable<Student>`) di ricevere direttamente un parametro di tipo `Student` nel metodo `compareTo(Student other)`, eliminando controlli `instanceof` e `downcast` forzati.

- **Iterable<E> e Iterator<E>:**

```
1 public interface Iterable<E> {
2     Iterator<E> iterator();
3 }
4 public interface Iterator<E> {
5     boolean hasNext();
6     E next();
7 }
```

Permette a costrutti come il ciclo `for-each` di iterare in modo fortemente tipizzato senza alcun cast:

```
1 for (Student student : classRoom) {
2     System.out.println(student.toString());
3 }
```

0.1.5 1.5 Il Diamond Operator <> (Slide 14)

Introdotta in Java 7, evita la ridondanza tra la dichiarazione del tipo di riferimento e l'istanziazione:

```
1 // Java 5 e 6:
2 Pair<Integer> intPair = new Pair<Integer>(2, 4);
3
4 // Java 7+:
5 Pair<Integer> intPair = new Pair<>(2, 4); // Il compilatore inferisce il tipo Integer dai membri a sinistra
```

0.1.6 1.6 Bounded Type Parameters: Vincolare i Parametri di Tipo (Slide 15-17)

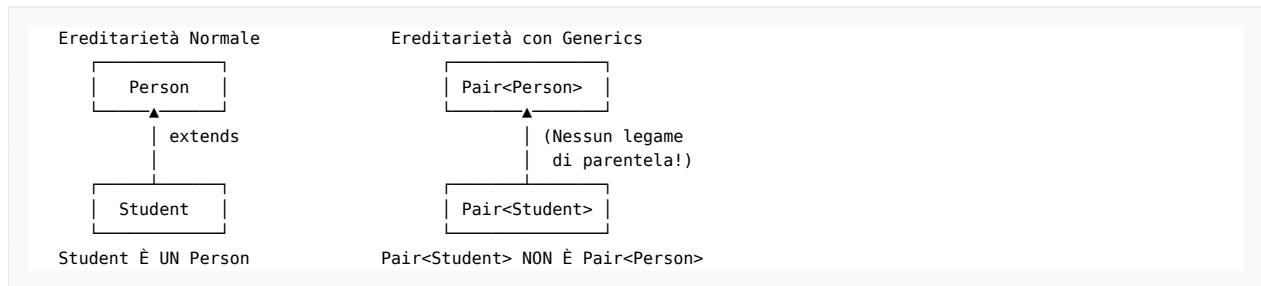
- **Unbounded Type Parameter (Slide 15):** Se dichiariamo `public class Pair<T>`, `T` è privo di vincoli. Il compilatore applica il meccanismo del **Type Erasure** assumendo che `T` sia un generico `Object`. Di conseguenza, su un oggetto di tipo `T` è possibile invocare **esclusivamente i metodi definiti in `Object`** (`equals`, `hashCode`, `toString`, `getClass`). Se tentiamo di chiamare `first.getName()`, il compilatore rigetta il codice con errore: `cannot find symbol: method getName() in T`.
- **Upper Bound con `extends` (Slide 16-17):** Possiamo imporre che `T` sia un sottotipo di una determinata classe o implementi una o più interfacce:

```
1 public class Pair<T extends Person> {
2     private T first, second;
3     // ...
4     public String getFirstName() {
5         return first.getName(); // PERFETTAMENTE LEGALE: T è garantito essere almeno una Person!
6     }
7 }
```

- **Vincoli multipli:** `<T extends ClassA & InterfaceB & InterfaceC>`. La classe (se presente) deve comparire per prima, seguita dalle interfacce separate da `&`.
- *Nota terminologica della Slide 16:* La slide cita `classClazz<T super S>`, ma in Java a livello di dichiarazione di classe generica è ammesso solo `extends`. La parola chiave `super` è utilizzabile **esclusivamente nelle Wildcards** (che vediamo subito sotto).

0.1.7 1.7 Ereditarietà e Generics: La Trappola dell'Invarianza (Slide 18-20)

Questo è uno dei punti più critici dell'intero corso e delle domande d'esame.



- **Invarianza dei Generics (Slide 19):** Anche se `Student` è sottotipo di `Person`, `Pair<Student>` **NON** è un sottotipo di `Pair<Person>`!

```

1 Student s = new Student("Sam", 123);
2 Person p = s; // OK: principio di sostituzione di Liskov
3
4 Pair<Student> pairS = new Pair<>(s, new Student("Paul", 456));
5 Pair<Person> pairP = pairS; // COMPILE-TIME ERROR: Incompatible types!
  
```

- **Perché Java vieta questo assegnamento?:** Se `pairP = pairS` fosse consentito, potremmo eseguire:

```

1 pairP.setFirst(new Professor("Mario", 999)); // Legale su Pair<Person>, poiché un Professor è una Person!
  
```

Ma poiché `pairP` e `pairS` punterebbero allo stesso identico oggetto nell'Heap, `pairS` (che pretende di contenere solo `Student`) si troverebbe a contenere un `Professor`, distruggendo la garanzia di tipo a runtime!

- **Il Contrasto con gli Array (Array Covariance, Slide 20):** In Java gli array sono storicamente **covarianti**:

```

1 Student[] arrS = { new Student("Sam", 123) };
2 Person[] arrP = arrS; // COMPILA PERFETTAMENTE!
3 arrP[0] = new Professor("Mario", 999); // BOOM! A runtime genera: java.lang.ArrayStoreException
  
```

La covarianza degli array è universalmente riconosciuta come una debolezza storica di Java (introdotta in Java 1.0 prima dell'avvento dei generics). I Generics sono stati resi **invarianti** proprio per risolvere questa falla a tempo di compilazione.

0.1.8 1.8 Wildcards: `<?>`, `<? extends T>`, `<? super T>` (Slide 21-23)

Per recuperare la flessibilità polimorfica senza rinunciare alla Type Safety, Java ha introdotto il carattere jolly (Wildcard) `?`.

1. Unbounded Wildcard (<?>, Slide 21):

- Rappresenta un tipo totalmente sconosciuto:

```
1 Pair<?> pairUnknown = pairS; // Legale!
```

- Possiamo **leggere** elementi, ma il loro tipo a compile-time è unicamente `Object`.
- **Non possiamo scrivere o aggiungere nulla** (tranne il letterale `null`), perché non sappiamo quale sia il tipo effettivo accettabile.

2. Upper-Bounded Wildcard (<? extends T>, Slide 22-23):

- Accetta `T` o qualsiasi sottoclasse di `T`:

```
1 Pair<? extends Person> pairP = pairS; // Legale! Student extends Person
```

- Ora possiamo invocare in sicurezza tutti i metodi di `Person`:

```
1 System.out.println(pairP.getFirst().getName()); // OK!
```

- **Sintassi pulita nei metodi:** Invece di dichiarare un parametro formale di tipo esplicito a livello di metodo:

```
1 <T extends Person> void printPair(Pair<T> p) { ... }
```

possiamo scrivere in modo molto più compatto e leggibile:

```
1 void printPair(Pair<? extends Person> p) { ... }
```

3. Il Principio Guida PECS (*Producer Extends, Consumer Super*):

- Se la struttura generica funge da **produttore** di dati (vogliamo solo *leggere* dati di tipo `T`), si usa `<? extends T>`.
- Se la struttura funge da **consumatore** di dati (vogliamo *scrivere/aggiungere* elementi di tipo `T`), si usa `<? super T>`.

0.2 2. Analisi Dettagliata del Codice (Lezione15/MavenDate)

Tra la Lezione 14 e la Lezione 15, il prof. Pasqua compie due modifiche sostanziali: 1. Implementa `hashCode()` in `Date.java`. 2. Riscrive ed espande `MainDate.java` per integrare la libreria delle **Java Collections** (`List`, `Set`, `Map`), il contratto `equals/hashCode`, e i meccanismi di ordinamento con `Comparable` e `Comparator` (usando classi anonime e `lambdas`).

0.2.1 2.1 Il Contratto `equals` / `hashCode` in `Date.java`

Nel passaggio da Lezione 14 a Lezione 15 viene aggiunto a `Date`:

```
1 @Override
2 public int hashCode() {
3     return (day + month) * year;
4 }
```

Perché questo metodo è vitale e non opzionale? In Java vige la regola ferrea del contratto di `Object`:
 > Se due oggetti sono considerati uguali dal metodo `equals(Object)` (`a.equals(b) == true`), allora le chiavi a `hashCode()` su entrambi gli oggetti DEVONO restituire lo stesso valore intero (`a.hashCode() == b.hashCode()`).

Se si fa l'override di `equals` senza fare l'override di `hashCode`, strutture dati basate su hash come `HashSet` e `HashMap` falliscono miseramente: considerano due date identiche come memorizzate in *bucket* differenti, inserendo duplicati o non trovando chiavi presenti.

Nel codice di `MainDate.java`:

```
1 AmericanDate usD1 = new AmericanDate(7, 11, 2025);
2 AmericanDate usD2 = new AmericanDate(7, 11, 2025);
3 System.out.println("usD1 ?= usD2: " + usD1.equals(usD2)); // true
4 System.out.println("hash(usD1): " + usD1.hashCode());      // 36450
5 System.out.println("hash(usD2): " + usD2.hashCode());      // 36450
```

Calcolo: $(7 + 11) \times 2025 = 18 \times 2025 = 36450$. Entrambi gli oggetti hanno identico hash code!

0.2.2 2.2 List vs Set: Accettazione vs Rifiuto dei Duplicati in `MainDate.java`

```
1 // 1. Creazione di una List (ammette elementi duplicati e mantiene l'ordine di inserimento)
2 List<Date> dateList = new ArrayList<>();
3 dateList.add(new AmericanDate(7, 11, 2025));
4 dateList.add(new Date(9, 11, 2025));
5 dateList.add(new Timestamp(0, 0, 0, 8, 11, 2025));
6 dateList.add(new AmericanDate(7, 11, 2025)); // Duplicato!
7 System.out.println("list(" + dateList.size() + "): " + dateList.toString());
8
9 // 2. Creazione di un Set (insieme matematico: nessun duplicato consentito)
10 Set<Date> dateSet = new HashSet<>();
11 dateSet.add(new AmericanDate(7, 11, 2025));
12 dateSet.add(new Date(9, 11, 2025));
13 dateSet.add(new Timestamp(0, 0, 0, 8, 11, 2025));
14 dateSet.add(new AmericanDate(7, 11, 2025)); // Duplicato: viene scartato silenziosamente!
15 System.out.println("set(" + dateSet.size() + "): " + dateSet.toString());
```

• Risultato a runtime:

- `dateList.size()` vale **4** (contiene entrambe le istanze di 7/11/2025).
- `dateSet.size()` vale **3** (la seconda istanza è stata rifiutata perché `equals` ha restituito `true` e l'`hashCode` coincideva).

0.2.3 2.3 Map<K, V> e Lambdas Registrate Dinamicamente in `MainDate.java`

```
1 Map<String, FormattedDateConverter> converter = new HashMap<>();
2 // Registrazione di strategie di conversione tramite espressioni lambda
3 converter.put("usToIt", d -> new ItalianDate(d.getDay(), d.getMonth(), d.getYear()));
4 converter.put("itToUs", d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear()));
5
6 FormattedDate usDate = new AmericanDate(1, 1, 1970);
7 FormattedDate itDate = converter.get("usToIt").convert(usDate);
8 System.out.println(itDate.prettyPrint()); // Stampa "1 gennaio 1970"
9
10 for (String k : converter.keySet())
11     System.out.println(converter.get(k).toString());
```

- **Finezza di esame sul `toString()` delle Lambdas:** Quando si invoca `toString()` su un oggetto lambda in Java, la JVM non stampa il codice sorgente, bensì il descrittore sintetico della classe generata a runtime (invokedynamic): `it.oop.ui.MainDate$$Lambda/0x...` (la notazione interna con i due dollari `$$Lambda` è la convenzione standard con cui la JVM battezza le classi sintetiche generate dinamicamente).

0.2.4 2.4 Ordinamento: Comparable vs Comparator (Slide 11 & MainDate.java)

In Java esistono due modi per ordinare oggetti: 1. **Ordinamento Naturale con `Comparable<T>`:** * Definito all'interno della classe stessa tramite `compareTo(T other)`. * `Date` implementa `Comparable<Date>`: `java Collections.sort(dateList)` // Funziona direttamente! La lista disordinata `[11/7/2025, y2025m11d9, y2025m11d8, 11/7/2025]` viene riordinata cronologicamente: `[11/7/2025, 11/7/2025, y2025m11d8[00:00:00], y2025m11d9]`.

2. Ordinamento Personalizzato con `Comparator<T>`:

- Consideriamo `DateInterval`: eredita da `OrderedPair<Date>`, ma **NON implementa `Comparable<DateInterval>`!**
- Se si prova a chiamare:

```
1 // Collections.sort(intvList); // COMPILE-TIME ERROR:
2 // no suitable method found for sort(List<DateInterval>)
3 // reason: cannot infer type-variable(s) T (DateInterval is not Comparable)
```

- Per risolvere il problema, il prof. Pasqua applica due stili per fornire un `Comparator<DateInterval>` esterno:

– **Stile 1: Classe Anonima** (ordinamento per data di inizio left crescente):

```
1 Collections.sort(intvList, new Comparator<DateInterval>() {
2     @Override
3     public int compare(DateInterval di1, DateInterval di2) {
4         return di1.getLeft().compareTo(di2.getLeft());
5     }
6 });
```

– **Stile 2: Espressione Lambda** (ordinamento per data di fine right decrescente):

```
1 Collections.sort(intvList, (di1, di2) -> di2.getRight().compareTo(di1.getRight()));
```

0.3 3. Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel progetto [Lezione15/MavenDate](#):

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainDate"
```

Output ottenuto:

```
1 y2025m9d10
2 y2025m1d1
3 00:00:00
4 true
5 OOP exam on 2 febbraio 2026
6 date pair: [y2025m2d1 .. y2025m1d1]
7 Pair not ordered!
8 OOP exam on 2 febbraio 2026
9 list(4): [11/7/2025, y2025m11d9, y2025m11d8[00:00:00], 11/7/2025]
10 set(3): [11/7/2025, y2025m11d9, y2025m11d8[00:00:00]]
11 usD1 ?= usD2: true
12 hash(usD1): 36450
```

```
13 hash(usD2): 36450
14 1 gennaio 1970
15 it.oop.ui.MainDate$$Lambda/0x0000000008f383b48@21043ee9
16 it.oop.ui.MainDate$$Lambda/0x0000000008f383920@6dc5f73f
17 [11/7/2025, 11/7/2025, y2025m11d8[00:00:00], y2025m11d9]
18 [[y2025m1d1 .. y2025m1d20], [y2025m1d10 .. y2025m1d31]]
19 [[y2025m1d10 .. y2025m1d31], [y2025m1d1 .. y2025m1d20]]
```

0.3.1 Decodifica puntuale dell'output:

1. `list(4): [...]` vs `set(3): [...]` → Dimostrazione pratica che `HashSet` elimina il duplicato `11/7/2025` grazie al corretto funzionamento congiunto di `equals()` e `hashCode()`.
2. `hash(usD1): 36450` e `hash(usD2): 36450` → Rispetto formale del contratto di `Object`.
3. `1 gennaio 1970` → La mappa `converter` recupera il `converter "usToIt"` memorizzato come `lambda` e trasforma `1/1/1970` in formato italiano.
4. `it.oop.ui.MainDate$$Lambda/...` → Nome sintetico dell'oggetto funzionale generato dalla JVM a runtime (in cui il doppio dollaro `$$Lambda` rappresenta il discriminatore standard del bytecode).
5. `[11/7/2025, 11/7/2025, y2025m11d8[00:00:00], y2025m11d9]` → Lista ordinata secondo l'ordinamento naturale di `Date` (7 nov, poi 8 nov, poi 9 nov).
6. `[[y2025m1d1 .. y2025m1d20], [y2025m1d10 .. y2025m1d31]]` → Intervalli ordinati per data d'inizio crescente (1 gen prima del 10 gen) tramite `Comparator` in classe anonima.
7. `[[y2025m1d10 .. y2025m1d31], [y2025m1d1 .. y2025m1d20]]` → Intervalli ordinati per data di fine decrescente (31 gen prima del 20 gen) tramite `Comparator` in espressione `lambda`.

Dimmi “**vai**” per procedere con il **Blocco 15** (Lezione 16 / Slide T16 — *Collections Framework*, gerarchia completa di `List`, `Set`, `Queue`, `Map`, complessità asintotiche, iteratori e `diff` in *Lezione16/MavenDate*).